

Inter-Service Communication

Sync vs Async

Learning Objectives

After this section, students will be able to:

- Explain the difference between Synchronous and Asynchronous communication.
 - Decide when to use each communication style.
 - Understand the difference between Blocking and Non-Blocking execution.
 - Choose between OpenFeign, WebClient, and Kafka.
-

The Big Picture

In a Microservices architecture, services constantly communicate with each other.

Example:

Order Service



Inventory Service

Payment Service

Notification Service

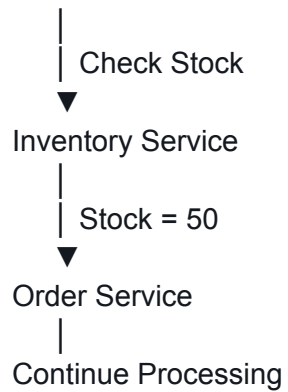
There are **two communication styles**:

1. Synchronous Communication
 2. Asynchronous Communication
-

1) Synchronous Communication (Sync)

The caller sends a request and **waits** for the response before continuing.

Order Service



Think of it like making a phone call.

You ask a question...

You wait...

Then continue.

Characteristics

- Immediate response
- Request/Response
- Caller waits
- Easier to understand
- Can increase latency

Typical Technologies

- OpenFeign
- WebClient
- RestTemplate (deprecated)

Example

Order Service

↓

Check Inventory

↓

Inventory replies



Create Order

Without the inventory response, the order cannot continue.

When should we use Sync?

Use Sync when the answer is required immediately.

Examples

✓ Check product stock

✓ Validate coupon

✓ Verify customer

✓ Charge payment

2) Asynchronous Communication (Async)

The caller sends a message and **does not wait**.

Order Service

Publish Event



Kafka



Notification Service



Send Email

Order Service continues immediately.

Think of it like sending an email.

You don't wait for the recipient to reply.

Characteristics

- No waiting
 - Better scalability
 - Loosely coupled
 - Event Driven
 - Eventually Consistent
-

Typical Technologies

- Kafka
 - RabbitMQ
 - Spring Cloud Bus
-

Example

Customer places an order

↓

Order Created

↓

Publish OrderPlaced Event

↓

Notification Service

↓

Send Email

Even if email takes 5 seconds...

The customer already received

Order Created Successfully

Sync vs Async

Feature	Sync	Async
Wait for response	✓ Yes	✗ No
Immediate result	✓ Yes	✗ No
Coupling	Higher	Lower
Performance	Lower	Higher
Complexity	Easier	More complex
Typical Tools	OpenFeign, WebClient	Kafka, RabbitMQ

Decision Framework

Ask yourself three questions.

Question 1

Do I need the answer immediately?

YES

↓

Use Sync

NO

↓

Use Async

Question 2

Can this operation happen later?

YES

↓

Async

NO

↓

Sync

Question 3

Is eventual consistency acceptable?

YES

↓

Async

NO

↓

Sync

Real Platform Examples

Scenario	Communication
Order → Inventory	Sync
Order → Payment	Sync
Order → Notification	Async

Order → Analytics Async

Order → Audit Log Async

Blocking vs Non-Blocking

Now comes another concept.

Many students confuse this with Sync/Async.

They are **NOT the same thing**.

Blocking

A thread waits until the response arrives.

Request

↓

Thread waits

↓

Response

↓

Continue

The thread is occupied during the waiting time.

Non-Blocking

The thread sends the request and is immediately released.

When the response arrives later, another callback continues the work.

Request

↓

Thread released

...

Response arrives

↓

Continue

No thread is blocked while waiting.

Important

Blocking / Non-Blocking is about **threads**.

Sync / Async is about **communication style**.

These are different concepts.

Four Possible Combinations

Communication	Thread Model	Possible?
Sync + Blocking	✓ Yes	Most common
Sync + Non-Blocking	✓ Yes	WebClient
Async + Blocking	⚠ Rare	
Async + Non-Blocking	✓ Kafka, Reactive	

OpenFeign

OpenFeign is the recommended tool for synchronous communication.

Order Service

↓

OpenFeign

↓

Inventory Service

↓

Return Response

Very simple.

Looks like calling a Java interface.

RestTemplate

Older HTTP client.

Requires writing every HTTP request manually.

Deprecated since Spring Framework 6.

Do not use it in new projects.

WebClient

WebClient is the modern HTTP client.

It supports:

- Synchronous communication
- Reactive programming
- Non-Blocking execution

Many developers think:

WebClient

↓

Async only

This is incorrect.

WebClient can be used synchronously.

Example

WebClient

↓

Inventory Service

↓

Wait for response

↓

Continue

It is still **Synchronous Communication**.

The difference is that the thread is **not blocked** while waiting.

CompletableFuture

CompletableFuture is a Java feature.

It allows asynchronous programming inside your application.

Call Service A

Call Service B

Call Service C

↓

Wait for all

↓

Continue

Useful when multiple independent calls can execute in parallel.

It is **not** a messaging system.

Kafka

Kafka is completely different.

Order Service

↓

Publish Event

↓

Kafka

↓

Inventory Service

↓

Notification Service

↓

Analytics Service

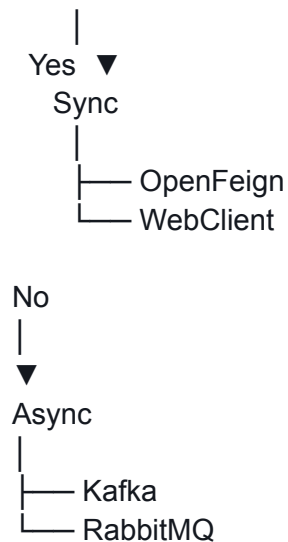
No service waits for another.

Which One Will We Use in This Course?

Session	Technology	Purpose
Session 6	OpenFeign	Synchronous communication
Session 7	Kafka	Asynchronous communication
Session 7	CompletableFuture	Parallel asynchronous tasks inside a service

Final Decision Tree

Need the response immediately?



Key Takeaways

- **Sync** means the caller waits for the response.
- **Async** means the caller continues without waiting.
- **Blocking** means a thread waits.
- **Non-Blocking** means the thread is released while waiting.
- **OpenFeign** is the recommended choice for synchronous service-to-service communication.
- **WebClient** is a modern non-blocking HTTP client and can be used for synchronous communication.
- **Kafka** is used for asynchronous, event-driven communication between services.
- **CompletableFuture** enables asynchronous execution within a single service and is not a replacement for Kafka.